

1. Scopul lucrarii	
	• studiul generatorului de analizoare sintactice YACC • efectuarea unor exercitii practice de utilizare a acestuia
2. Notiuni teoretice	
2.1. Introducere	
	Pornind de la o gramatica independenta de context, YACC-ul genereaza un set de tabele pentru un automat de analiza LR(1). YACC-ul se apeleaza astfel: YACC [-vlt] [-o nume_fis.c] [-d ytab.h] fisier_sursa_YACC.y , unde: • -v genereaza fisierul y.out, care descrie tabelele de analiza si raporteaza conflictele generate de ambiguitatile din gramatica. • -d genereaza fisierul ytab.h cu instructiunile #define ce asociaza codurile token fixate de YACC cu numele de token-i declarate de utilizator. Aceasta optiune permite unor fisiere sursa, altele decit ytab.c (implicit) respectiv nume_fis.c (optiunea -o) sa acceseze codurile token. • -l produce cod fara constructii #line, in nume_fis.c/ytab.c. Optiunea trebuie folosita doar dupa ce gramatica si actiunile asociate sint depanate. • -t include cod de depanare runtime cind nume_fis.c/ytab.c e compilat. Deoarece numele de fisiere sint fixe, cel mult un proces YACC poate fi activ intr-un director la un moment dat. Funcția produsă de YACC utilizează o rutină (analizor lexical) furnizată de utilizator pentru a returna următorul atom = simbol terminal = token de la intrare. Astfel, utilizatorul își poate specifica intrarea ca și caractere individuale, sau ca și construcții elaborate, cum ar fi constantele numerice sau identificatorii. Rutina furnizată de utilizator poate de asemenea să trateze și obiecte cum sunt comentariile sau comenzile de continuare. Utilizatorul YACC-ului pregătește o specificație a procesului de intrare, ce include reguli ce descriu structura de intrare, codul de executat la recunoașterea acestor reguli și o rutină de nivel coborât pentru a efectua procesul de intrare. Clasa specificațiilor acceptate e foarte generală: gramatici LALR(1) cu reguli de dezambiguizare. Token-ii sunt organizați pe baza regulilor gramaticale, și, cînd una dintre aceste reguli e potrivită, e invocată acțiunea corespunzătoare. Acțiunile au posibilitatea de a returna valori și de a utiliza valorile altor acțiuni. YACC-ul e scris într-un C portabil, acțiunile și rutinele de ieșire de asemenea, și multe din convențiile sintactice ale YACC-ului urmează C-ul. O parte importantă a procesului de intrare este realizată de analizorul lexical, care citește fluxul de intrare, recunoaște structurile de nivel coborât, token-ii, și îi comunică analizorului sintactic. Din motive istorice, o structură recunoscută de analizorul lexical se numește simbol terminal, iar o structură recunoscută de analizorul sintactic simbol non-terminal. Pentru a evita confuziile, terminalele vor fi referite ca token-i. O problemă serioasă constă în a decide dacă anumite structuri să fie recunoscute de analizorul lexical sau de reguli gramaticale. De exemplu, dacă am folosi regulile: <pre> month_name : 'J' 'a' 'n' ; month_name : 'F' 'e' 'b' ; month_name : 'D' 'e' 'c' ; </pre> , atunci analizorul lexical ar trebui să recunoască doar litere individuale, iar "month_name" ar fi un

	simbol non-terminal. Asemenea reguli de nivel coborit tind insa sa risipeasca timp si spatiu si pot complica specificatia dincolo de capacitatea de solutionare a YACC-ului. In mod normal, analizorul lexical va recunoaste aceste "nume de luna" si va returna o informatie ca a fost citit un "nume de luna" ("month-name"). In acest caz "month_name" va fi un token. Caracterele literale cum ar fi ',' trebuie si ele trecute prin analizorul lexical si sint de asemenea considerate token-i. Intrarea poate sa nu corespunda specificatiilor. In unele cazuri YACC-ul nu reuseste sa produca un analizor sintactic. De exemplu, specificatiile pot fi contradictorii sau ele pot cere un mecanism de recunoastere mai puternic decit cel disponibil in YACC. In prima situatie e vorba de erori, iar cea de-a doua poate fi in general corectata facind analizorul lexical mai puternic sau rescriind o parte din regulile gramaticale.
--	--

2.2. Specificatii de baza

	Identificatorii ce apar in specificatii refera fie token-i fie simboluri non-terminale. YACC-ul cere ca numele de token-i sa fie declarate. In plus, adesea e de dorit sa se includa analizorul lexical ca parte a fisierului de specificatii, care poate sa includa si alte programe. Un fisier cu specificatii consta din 3 sectiuni separate prin %% : declaratii %% reguli %% programe Sectiunea de declaratii poate fi vida, iar daca sectiunea de programe e omisa, poate fi omis si al doilea delimitator %%. Astfel, cea mai scurta specificatie YACC este: %% reguli "Blanc"-urile, "tab"-urile si "newline"-urile sint ignorate si ele nu pot sa apara in nume sau simboluri multi-caracter rezervate. Comentariile pot sa apara oriunde e legal un nume si ele trebuie incluse intre /* ... */ . Sectiunea de reguli cuprinde una sau mai multe reguli gramaticale, de forma: A: CORP;, unde A reprezinta un nume de non-terminal, iar CORP reprezinta o secventa de zero sau mai multe nume si literale. Doua puncte, si punct si virgula, sint punctuatii YACC. Numele pot avea orice lungime si pot fi alcatuite din litere, punct, "underscore" si cifre, dar trebuie sa inceapa cu litera. Literele mici sint distincte de cele mari. Numele folosite in corpul unei reguli gramaticale pot reprezenta token-i sau non-terminale. Un literal consta dintr-un caracter inclus intre apostroafe. Ca in C, "backslash"-ul, e un caracter "escape". Toate secventele "escape" din C sint recunoscute: '\n' = "newline", '\r' = "return", '\'' = apostrof, '\\' = "backslash", '\t' = "tab", '\b' = "backspace", '\f' = "formfeed", '\xxx' = xxx in octal. Caracterul NUL ('\0' sau 0) nu trebuie folosit niciodata in regulile gramaticale. Daca exista mai multe reguli gramaticale cu aceeasi parte stinga, se poate folosi bara verticala ' ' pentru a evita rescrierea. In plus, punct si virgula de la sfirsitul unei reguli pot fi eliminate cind urmeaza o bara verticala. Astfel regulile: A : B C D ; A : E F ; A : G ; pot fi scrise si ca: A : B C D E F G ;
--	--

LAB-7-LFT- INSTRUMENTUL YACC

	Nu e obligatoriu ca toate regulile gramaticale cu aceeasi parte stinga sa apara impreuna, dar e recomandabil. Daca un simbol non-terminal recunoaste vid, aceasta se scrie de exemplu: empty : ; Numele ce reprezinta token-i trebuie declarate, si aceasta se face cel mai simplu folosind cuvintul cheie %token : % token nume1 nume2 nume3 ... ,in sectiunea de declaratii. Orice nume nedefinit in sectiunea de declaratii e presupus a reprezenta un non-terminal. Orice simbol non-terminal trebuie sa apara in partea stinga a cel putin unei reguli. Dintre simbolurile non-terminale, simbolul de start ("start simbol") are o importanta particulara. Analizorul sintactic recunoaste de fapt simbolul de start. Acest simbol reprezinta cea mai larga si mai generala structura descrisa de regulile gramaticale. Simbolul de start este considerat implicit a fi partea stinga a primei reguli gramaticale. E de dorit insa sa se declare simbolul de start in mod explicit, in sectiunea de declaratii, folosind cuvintul cheie %start : %start simbol_de_start Sfirsitul intrarii e semnalat de un token special, numit marcator de sfirsit ("end-marker"). Daca token-ii cititi pina la "end-marker" formeaza o structura ce se potriveste cu simbolul de start, dupa ce e intilnit "end-marker"-ul analizorul revine la apelantul sau si accepta intrarea. Daca marcatorul de sfirsit e "vazut" in orice alt context, exista o eroare. Analizorul lexical are sarcina de a returna "end-marker"-ul. In mod normal acesta este o valoare de I/E ca "end-of-file" sau "end-of-record".
--	--

2.3. Actiuni YACC

	Utilizatorul poate asocia fiecarei reguli gramaticale actiuni de realizat cind respectiva regula e recunoscuta in fluxul de intrare. Aceste actiuni pot returna valori si pot obtine valorile returnate de actiunile precedente. Mai mult, analizorul lexical poate returna valori pentru token-i. O actiune, e o secventa de instructiuni C. Astfel: A : '(' B '){ hello(1, "abc");} XXX : YYY ZZZ { printf ("a message\n"); flag=25;} sint exemple de reguli gramaticale cu actiuni. Simbolul dolar, '\$', e un semnal special pentru YACC. Pentru a returna o valoare, actiunea seteaza pseudo-variabila \$\$ la acea valoare. De exemplu, o actiune care nu face altceva decit sa returneze valoarea 1, e: { \$\$= 1;} Pentru a obtine valori returnate de actiuni precedente sau de analizorul lexical, se pot folosi pseudo-variabilele: \$1, \$2, ... , care refera valorile returnate de componentele din partea dreapta a regulii curente, citind de la stinga la dreapta. Astfel, pentru: A : B C D ; , \$2 are valoarea returnata de C, iar \$3 valoarea returnata de D. Sa consideram: expr : '(' expr '); Valoarea returnata de aceasta regula e de obicei valoarea lui expr dintre paranteze. Acest lucru poate fi precizat astfel: expr : '(' expr ')' { \$\$=expr;} Implicit, valoarea unei reguli e valoarea primului sau element, \$1. Astfel, regulile gramaticale de forma: A : B; nu au nevoie de actiunea explicita { \$\$=\$1; }. In exemplele de mai sus, toate actiunile apar la sfarsitul regulilor. Uneori se doreste obtinerea controlului inainte ca o regula sa fie complet analizata. YACC-ul permite scrierea unei actiuni si in interiorul unei reguli. Aceasta actiune poate returna o valoare, accesibila actiunilor de la dreapta sa, si ea poate la rindul ei accesa valorile returnate de simbolurile de la stinga sa. Astfel, in regula: A : B
--	---

	<pre> { \$\$=1;} C { x=\$2; y=\$3;} ; </pre> , efectul consta in setarea lui x la 1 si a lui y la valoarea returnata de C. Actiunile ce nu termina o regula, sint tratate de YACC prin manipularea unui nou nume de simbol non-terminal si a unei noi reguli ce asociaza acest nume sirului vid. Actiunea interioara este actiunea "comutata" la recunoasterea acestei reguli adaugate. YACC-ul trateaza exemplul de mai sus astfel: <pre> \$ACT : /* empty */ { \$\$=1;} ; A : B \$ACT C { x=\$2; y=\$3;} ; </pre> In multe aplicatii, iesirea nu e realizata direct de actiuni. Se construiesc mai degraba o structura de date (cum ar fi un arbore de derivare) in memorie, si se aplica transformari respectivei structuri inainte de a se genera iesirea. Arborii de derivare sunt usor de construit, daca sunt date rutinele necesare pentru a construi si mentine structura arborescenta dorita. Sa presupunem ca exista o functie node(), scrisa astfel incit apelul node (L,n1,n2), creeaza un nod cu eticheta L, descendenti n1 si n2, si returneaza indexul nodului nou creat. Atunci arborele de derivare poate fi construit cu actiuni ca: <pre> expr : expr '+' expr { \$\$= node('+', \$1, \$3); } </pre> Utilizatorul poate defini alte variabile pentru a fi folosite de actiuni. Declaratiile si definitiile pot sa apara in sectiunea de declaratii, incluse intre marcatorii %{ si %} . Aceste declaratii si definitii au un domeniu global, deci sint cunoscute atat de instructiunile actiune cit si de analizorului lexical. De exemplu: <pre> %{ int var1 = 0; %} </pre> , plasata in sectiunea de declaratii face variabila var1 accesibila tuturor actiunilor. Analizorul YACC foloseste numai identificatori ce incep cu 'yy', motiv pentru care utilizatorul trebuie sa evite astfel de nume.
--	---

2.4. Analiza lexicala

- Utilizatorul trebuie sa furnizeze un analizor lexical care sa citeasca fluxul de intrare si sa comunice token-ii (cu valori, daca se doreste) analizorului sintactic.
- Analizorul lexical e o functie ce returneaza intregi, numita yylex().
- Functia returneaza numarul terminalului ("token-number"), reprezentind tipul de token citit.
- Daca exista o valoare asociata cu acel token, ea trebuie atribuita variabilei externe yylval.
- Analizorul sintactic si cel lexical trebuie sa se "inteleaga" asupra acestor numere de token, pentru ca ele sa poata comunica corect.
- Numerele pot fi alese de YACC sau de utilizator.
- In ambele cazuri, se foloseste mecanismul #define din C pentru a permite analizorului lexical sa returneze simbolic aceste numere.
- Sa presupunem ca numele de token DIGIT a fost definit in sectiunea de declaratii a fisierului de specificatii YACC.
- Portiunea relevanta a analizorului lexical ar putea arata astfel:

```

yylex()
{
extern int yylval; int c;

```

LAB-7-LFT- INSTRUMENTUL YACC

```

c=getchar();
switch(c){
case '0':
case '1':
.....
case '9' :
        yyval=c-'0';
        return(DIGIT);
}

```

Intentia este de a returna un cod de token DIGIT si valoarea sa numerica. Cind codul pentru analizorul lexical e plasat in sectiunea "programe" a fisierului de specificatii, identificatorul DIGIT va fi definit ca numarul de token asociat cu token-ul DIGIT.

Acest mecanism conduce la analizoare lexicale limpezi si usor de modificat. Singura capcana e necesitatea de a evita orice nume de token care e rezervat sau semnificativ in C sau in analizor.

De exemplu, utilizarea numelor de token if sau while va cauza aproape sigur dificultati mari la compilarea analizorului lexical.

Numele de token error e rezervat pentru tratarea erorilor si trebuie utilizat in mod corespunzator. Dupa cum s-a mentionat, numerele de token pot fi alese de YACC sau de utilizator. Implicit, ele sint alese de YACC.

Numarul de token implicit pentru un literal caracter este valoarea numerica a caracterului in setul de caractere local.

Celorlalte nume le sint asociate numere de token incepind cu 257.

Pentru a asigna un numar unui token, inclusiv literalelor, prima aparitie a numelui de token sau a literalului in sectiunea de declaratii poate fi imediat urmata de un intreg nenegativ. Acest intreg e tratat a fi numarul de token al numelui sau al literalului.

Numele si literalele ce nu sint definite prin acest mecanism isi pastreaza definitia lor implicita. E important ca toate numerele de token sa fie distincte.

Marcatorul de sfirsit ("end_marker"-ul) trebuie sa aiba numarul de token 0 (zero) sau negativ. Acest numar de token nu poate fi redefinit de utilizator.

Analizorul lexical trebuie sa fie pregatit sa returneze zero sau un numar negativ la intilnirea sfirsitului intrarii.

Un instrument util pentru constructia de analizoare lexicale este LEX-ul, care genereaza analizoare potrivite pentru a lucra in armonie cu analizoarele sintactice generate de YACC.

2.5. Modul de lucru al analizorului sintactic

- Analizorul generat de YACC pe baza specificatiilor date este relativ simplu si consta dintr-un automat finit cu stiva.
- Analizorul e capabil sa citeasca si sa memoreze simbolul de intrare urmator, numit token "look-ahead", sau "priveste-inainte".
- Starea curenta este intotdeauna cea din virful stivei. Starilor automatului le sint atribuite etichete (intregi mici sau "short int").
- Initial masina e in starea 0 (zero), stiva contine starea 0 si nici un token "look-ahead" nu a fost citit.
- Automatul are 4 actiuni la dispozitie: deplasare, reducere, acceptare, eroare. Analizorul lucreaza astfel:
 1. Pe baza starii curente, se decide daca trebuie citit inainte ("look-ahead") un token pentru a hotari actiunea de efectuat. Daca e nevoie de un nou token, e apelat yylex() pentru a-l obtine.

LAB-7-LFT- INSTRUMENTUL YACC

2. Folosind starea curenta si eventual token-ul "look-ahead", analizorul decide actiunea sa urmatoare si o efectueaza. Aceasta poate determina punerea pe stiva a unor stari (push), preluarea de pe stiva a unor stari (pop), sau consumarea sau doar examinarea inainte a tokenului urmator.

Intotdeauna cand are loc o deplasare, exista un token urmator. De exemplu, in starea 56 ar putea fi o actiune: **IF shift 34**, care spune ca in starea 56, daca token-ul "look-ahead" e IF, starea curenta, 56, e pusa pe stiva si starea 34 devine stare curenta si e pusa pe virful stivei. Token-ul urmator este consumat (sters) din intrare.

Reducerile pastreaza stiva spre a nu creste nelimitat, si sint realizate cind analizorul a vazut partea dreapta a unei reguli gramaticale si e pregatit sa anunte ca a vazut o instantiere a acesteia, inlocuind partea dreapta cu partea stinga. Poate fi necesar sa se consulte token-ul "look-ahead" pentru a decide daca sa se faca reducere, dar de obicei nu e necesar. De fapt, actiunea implicita (reprezentata printr-un punct) e adesea o reducere. Actiunile de reducere sint asociate cu reguli gramaticale individuale.

Regulilor gramaticale le sint asociate de asemenea intregi mici, ceea ce poate conduce la unele confuzii. Actiunea: **. reduce 19**, se refera la regula gramaticala 19, in timp ce actiunea: **IF shift 19**, se refera la starea 19.

Sa presupunem ca regula de redus este: **A : X Y Z ; .**
Pentru a reduce se elimina de pe stiva primele trei stari (intotdeauna se iau de pe stiva atatea stari cite simboluri sunt in partea dreapta a regulii). Aceste stari sint cele puse pe stiva cand s-a recunoscut **X, Y si Z**. Dupa ce s-au eliminat (pop) aceste stari, o noua stare e in virful stivei, si anume aceea in care era analizorul inainte de a incepe sa proceseze regula. Folosind aceasta stare si simbolul nonterminal A din stanga regulii, se realizeaza ceea ce ar fi o deplasare a lui A.
Se obtine o noua stare, care e pusa (push) pe stiva, si analiza continua.

Exista diferente semnificative intre procesarea simbolului nonterminal din partea stanga a unei reguli si o deplasare ordinara a unui token.

De aceea, deplasarea (consumarea) unui simbol nonterminal din stanga unei reguli este o actiune care se numeste goto.

Mai concret, token-ul urmator din intrare e consumat (sters) de o deplasare si nu e afectat de un goto. Starea din virful stivei dupa acea eliminarea (pop) a starilor contine o descriere ca:

A goto 20, starea 20 e pusa (push) pe stiva si devine stare curenta.

In concluzie, actiunea de reducere "intoarce timpul inapoi" pentru analizor, preluind starile de pe stiva pentru a se intoarce la starea in care a fost initial vazuta/banuita partea dreapta a regulii. Analizorul se comporta apoi ca si cind ar fi vazut partea stinga in acel moment. Daca partea dreapta a regulii este vida, nu se reia nici o stare de pe stiva, starea din virful stivei dupa acea "eliminare de stari" (pop) fiind chiar starea curenta.

Actiunea de reducere e importanta si in tratarea actiunilor si valorilor furnizate de utilizator.

Cind o regula este redusa, codul furnizat cu regula e executat inainte de modificarea stivei. In paralel cu stiva ce pastreaza starile, lucreaza o alta stiva ce pastreaza valorile returnate de analizorul lexical si de actiuni. Cand are loc o deplasare, variabila externa yylval e incarcata (push) pe stiva de valori. Dupa revenirea din codul utilizator, e efectuata reducerea. Cand e realizata actiunea goto, variabila externa yyval e incarcata (push) pe stiva valorilor.

Pseudo-variabilele \$1, \$2, ... refera de fapt stiva de valori. Celelalte doua actiuni sunt mult mai simple. Actiunea de acceptare indica faptul ca intreaga intrare a fost vazuta si ca aceasta se potriveste cu specificatia. Aceasta actiune apare cand token-ul urmator din intrare este "end-marker"-ul si indica faptul ca analizorul a terminat procesarea cu succes.

LAB-7-LFT- INSTRUMENTUL YACC

Actiunea de eroare reprezinta un "punct" din care nu se mai poate continua analiza conform specificatiei. Token-ii deja consumati din intrare impreuna cu token-ul urmator din intrare nu pot fi urmati de nimic ce ar constitui o intrare corecta.

Analizorul raporteaza o eroare si incearca sa refaca situatia si sa reia analiza.

Ca un exemplu, sa consideram specificatia :

```
%token      DING DONG DELL
%%
rhyme :      sound place
;
sound :      DING DONG
;
place :      DELL
;
```

Cind YACC-ul e invocat cu optiunea -v, el produce un fisier y.output, cu o descriere "umana" a analizorului. Fisierul y.output corespunzator specificatiei anterioare este:

```
state 0
    $accept :      _rhyme      $end
    DING shift 3
    . error
    rhyme goto 1
    sound goto 2

state 1
    $accept :      rhyme_$end
    $end accept
    . error

state 2
    rhyme :      sound_place
    DELL shift 5
    . error
    place goto 4

state 3
    sound :      DING_DONG
    DONG      shift 6
    . error

state 4
    rhyme :      sound place_ (1)
    . reduce 1

state 5
    place :      DELL_      (3)
    . reduce 3

state 6
    sound :      DING DONG_      (2)
    . reduce 2

unde:      shift = deplasare
```

LAB-7-LFT- INSTRUMENTUL YACC

reduce = reducere
accept = acceptare
error = eroare

De notat ca, pe langa actiunile pentru fiecare stare, exista o descriere a regulilor de analiza ce se proceseaza in fiecare stare. Caracterul "underscore" e utilizat pentru a indica ce a fost vazut si ce trebuie sa urmeze, in fiecare regula. Sa presupunem ca intrarea este: DING DONG DELL . Initial, starea curenta e 0. Analizorul trebuie sa analizeze intrarea pentru a alege una dintre actiunile disponibile in starea 0, deci primul token, DING, e citit, devenind token "look-ahead". In starea 0, actiunea pentru DING e shift 3, deci starea 3 este pusa pe stiva si token-ul "look-ahead" e sters . Starea 3 devine stare curenta. Urmatorul token, DONG e citit, devenind token "look-ahead".

In starea 3, actiunea pentru token-ul DONG este shift 6, deci starea 6 e pusa pe stiva si token-ul "look-ahead" e sters. Stiva contine acum 0, 3 si 6. In starea 6, fara a fi necesara consultarea "look-ahead", analizorul reduce prin regula 2: sound : DING DONG. Aceasta regula are doua simboluri in partea dreapta, deci doua stari, 6 si 3, sint luate de pe stiva, lasind in virful stivei starea 0.

Consultind descrierea starii 0 si cautind un goto pentru sound, se obtine: sound goto 2. Astfel starea 2 e pusa pe stiva, devenind stare curenta. In starea 2 e citit urmatorul token, DELL. Actiunea e shift 5, deci starea 5 e pusa pe stiva, care contine acum 0, 2 si 5 si token-ul "look-ahead" e sters. In starea 5, singura actiune e reducerea prin regula 3. Aceasta are un simbol in partea dreapta , deci se ia o stare de pe stiva, 5, raminand starea 2 in virful stivei. Actiunea goto in starea 2, pentru place (partea stinga a regulii 3), e starea 4. Acum, stiva contine 0, 2 si 4. In starea 4 singura actiune posibila e reducerea prin regula 1. Exista doua simboluri in dreapta, deci sint preluate de pe stiva doua stari, lasind starea 0 din nou pe virful stivei. In starea 0, exista un goto pentru rhyme , prin care analizorul intra in starea 1. In starea 1 e citit "end-marker"-ul (indicat prin '\$end' in fisierul y.output). Actiunea in starea 1 cind e "vazut" "end-marker"-ul , e acceptarea care termina analiza cu succes.

2.6. Ambiguitati si conflicte

Un set de reguli gramaticale este ambiguu, daca exista un sir de intrare ce poate fi structurat in doua sau mai multe feluri.

De exemplu regula: **expr : expr '-' expr** , spune ca putem forma o expresie aritmetica din alte doua expresii si un semn minus intre ele.

Din nefericire aceasta regula nu defineste complet felul in care ar putea fi structurate anumite intrari. De exemplu, pentru expr - expr - expr , regula permite structurarea fie ca (expr - expr) - expr , fie ca expr - (expr - expr) , prima fiind numita asociativitate stinga, iar a doua asociativitate dreapta.

Cind incearca sa construiasca analizorul, YACC-ul detecteaza asemenea ambiguitati.

Sa analizam problema cu care se confrunta el cind i se da o intrare de forma expr - expr - expr .

Cind e citita a doua expresie, intrarea vazuta, expr - expr , se potriveste cu partea dreapta a regulii gramaticale. Analizorul poate reduce aplicind aceasta regula si obtinind expr (partea stinga a regulii).

Analizorul poate citi atunci partea finala a intrarii: - expr , si poate reduce din nou. In acest caz s-ar folosi asociativitatea stinga. Dar, cind analizorul vede expr - expr , el ar putea amina aplicarea

imediată a regulii și să continue citirea până ar vedea: `expr - expr - expr`. Atunci ar putea aplica regula celor trei simboluri cele mai din dreapta, reducându-le la `expr`. S-ar obține `expr - expr`, și s-ar putea face încă o reducere. În acest caz s-ar folosi asociativitatea dreapta.

Astfel, la vederea `expr - expr` analizorul poate întreprinde două acțiuni legale: o deplasare sau o reducere, și nu are nici o indicație pentru alegere. Aceasta situație se numește conflict deplasare-reducere.

Se întâmplă ca analizorul să aibă de ales între două reduceri legale.

O astfel de situație este un conflict reducere-reducere. De notat însă că nu există niciodată conflicte deplasare-deplasare. YACC-ul reușește totuși să producă un analizor și când apar conflicte deplasare-reducere sau reducere-reducere. Dacă e necesar, el alege una dintre acțiunile posibile. O regulă ce descrie alegerea ce trebuie făcută într-o situație dată, se numește regulă de dezambiguizare. YACC-ul utilizează două reguli de dezambiguizare implicite:

1. Într-un conflict deplasare-reducere se alege deplasarea.
2. Într-un conflict reducere-reducere se alege regula gramaticală care apare prima (în secvența de intrare).

Cu alte cuvinte, regula 1 spune că reducerile sunt evitate când se poate face o deplasare, iar regula 2 dă un control relativ dur utilizatorului asupra comportamentului analizorului în situația unui conflict reducere-reducere, acestea din urmă trebuind evitate.

Conflictele pot să apară din erori de intrare sau de logică, sau din cauza regulilor gramaticale în cazul când ar fi necesar un analizor mai puternic decât poate YACC-ul să construiască.

Acțiunile din reguli pot de asemenea să genereze conflicte, dacă o acțiune trebuie efectuată înainte ca analizorul să poată fi sigur care regulă să fie recunoscută.

În aceste cazuri, aplicarea regulilor de dezambiguizare este nepotrivită și conduce la un analizor incorect. De aceea, YACC-ul raportează întotdeauna numărul de conflicte deplasare-reducere și reducere-reducere rezolvate prin regulile 1 și 2.

În general, dacă e posibil să se aplice reguli de dezambiguizare pentru a produce un analizor corect, e posibil și să se rescrie regulile gramaticale astfel încât pentru aceeași intrare, să nu existe conflicte. Din acest motiv, multe dintre generatoarele de analizoare din trecut au considerat conflictele ca erori fatale. Experiența autorilor a sugerat însă că această rescriere e oarecum nenaturală și produce analizoare mai lente. De aceea YACC-ul poate produce analizoare și în prezența conflictelor.

Ca exemplu, să considerăm un fragment ce descrie o construcție

```
'if_then_else' :      stat :  IF(' cond ') stat
                        |      IF(' cond ') stat ELSE stat
                        ;                                     , unde IF și ELSE
```

sunt token-uri, `cond` este un non-terminal ce descrie expresiile logice, iar `stat` este un simbol non-terminal ce descrie instrucțiunile. Prima regulă este un `if` simplu, iar a doua `if-else`. Aceste două reguli formează o construcție ambiguă, deoarece intrarea: `IF (C1) IF (C2) S1 ELSE S2`, poate fi structurată în două moduri: `IF (C1) { IF (C2) S1 } ELSE S2`, sau: `IF (C1) { IF (C2) S1 ELSE S2 }`.

- A doua interpretare, în care fiecare `ELSE` este asociat cu cel mai apropiat `IF`, este utilizată în majoritatea limbajelor de programare. Să considerăm situația în care analizorul a văzut: `IF (C1) IF (C2) S1`, și "privește la" `ELSE`. El ar putea reduce imediat ca `if` simplu pentru a obține `IF (C1) stat`, apoi să citească `ELSE S2`, și să reducă la `IF (C1) stat ELSE S2`, prin regula `if-else`. Aceasta ar conduce la prima grupare prezentată mai sus. Pe de altă parte, `ELSE` poate fi deplasat, se poate citi `S2`, după care partea din dreapta a `IF (C1) IF (C2) S1 ELSE S2`, poate fi redusă prin regula `if-else` pentru a obține: `IF (C1) stat`, care poate fi redusă prin regula `if` simplu. Aceasta ar conduce la a doua

LAB-7-LFT- INSTRUMENTUL YACC

grupare de mai sus.

In acest caz, regula 1 de dezambiguizare a YACC-ului ii spune analizorului sa execute deplasare, ceea ce conduce la gruparea dorita.

In general pot exista multe conflicte, fiecare fiind asociat cu un simbol de intrare si o anumita intrare citita pina in acel moment. Intrarea citita in prealabil e caracterizata de starea analizorului. Mesajele de conflict ale YACC-ului pot fi intelese examinind fisierul de iesire pentru optiunea (-v). De exemplu, iesirea corespunzand starii de conflict de mai sus ar fi:

23: shift/reduce conflict (shift 45, reduce 18) on ELSE

state 23

stat : IF (cond) stat_ (18)

stat : IF (cond) stat_ELSE stat

ELSE shift 45

. reduce 18

Prima linie descrie conflictul, dind starea si simbolul de la intrare.

Urmeaza descrierea starii ordinare, cu regulile gramaticale active si actiunile analizor.

Sa ne reamintim ca liniuta de subliniere (underscore) marcheaza portiunea din regula care a fost deja vazuta. Astfel, in starea 23 analizorul a vazut intrare corespunzind la IF (cond) stat , si sint aratate cele doua reguli gramaticale active in aceasta situatie.

Analizorul poate face doua actiuni:

[1] daca simbolul de intrare este ELSE, e posibil sa faca deplasare in starea 45, care va avea ca parte a descrierii sale linia stat : IF (cond) stat ELSE_stat , deoarece in aceasta stare ELSE ar fi deplasat;

[2] actiunea alternativa, descrisa de '.' va fi realizata daca simbolul de intrare nu e mentionat explicit in actiunile de mai sus, caz in care daca simbolul de intrare nu e ELSE , analizorul reduce prin regula gramaticala 18, stat : IF '(' cond ')' stat

Inca odata, e de notat ca numerele ce urmeaza lui "shift" se refera la stari, in timp ce numerele ce urmeaza lui 'reduce' sint numere de reguli gramaticale. In fisierul y.output, numerele de reguli sint scrise dupa acele reguli ce pot fi reduce. In multe stari, vor fi posibile cel mult actiuni de reducere si aceasta va fi comanda implicita. Utilizatorul care intalneste conflicte neasteptate deplasare-reducere va dori probabil sa vada iesirea pentru a decide daca actiunile implicite sint potrivite.

2.7. Precedenta

Exista o situatie in care regulile de rezolvare a conflictelor date mai sus nu sint suficiente, si anume in analiza expresiilor aritmetice. Multe dintre constructiile folosite pentru expresii aritmetice pot fi in mod natural descrise prin notiunea de nivele de precedenta ale operatorilor, impreuna cu informatie legata de asociativitatea stinga sau dreapta.

Deci, se pot folosi gramatici ambigue, dar cu reguli de dezambiguizare potrivite, pentru a crea analizoare mai rapide si mai usor de scris decit cele construite din gramatici neambigue.

Reguli gramaticale de forma:

expr : expr OP expr , si

expr : UNARY expr , scrise pentru toti operatorii unari si binari doriti, formeaza baza specificatiilor in acest caz.

Se creeaza o gramatica foarte ambigua, cu multe conflicte. Ca reguli de dezambiguizare, utilizatorul specifica precedenta tuturor operatorilor, si asociativitatea operatorilor binari.

Aceasta informatie este suficienta pentru ca YACC-ul sa rezolve conflictele de analiza in acord cu aceste reguli si sa construiasca un analizor care realizeaza precedentele si asociativitatile dorite.

Precedentele si asociativitatile sint atasate token-ilor in sectiunea de declaratii, printr-o serie de linii ce contin un cuvint cheie YACC: %left, %right sau %nonassoc, urmat de o lista de token-i. Toti token-ii de pe aceeasi linie vor fi tratati ca avind acelasi nivel de precedenta si asociativitate.

Liniile trebuie date in ordinea crescatoare a precedentei (a puterii de legare).

Astfel regulile:

%left '+' '-'

%left '*' '/' , descriu precedenta si asociativitatea celor 4 operatori aritmetici.

Plus si minus sint asociative la stinga si au precedenta mai mica decit stea si "slash", si ele asociative la stinga. %right e folosit pentru a descrie asociativitatea dreapta pentru operatori, iar %nonassoc e folosit pentru a descrie operatori cum ar fi .LT. din FORTRAN, care nu poate asocia cu el insusi. Astfel: A .LT. B .LT. C este ilegal.

De exemplu, specificatia:

```

%right      '='
%left  '+' '-'
%left  '*' '/'
%%
expr  :      expr '=' expr
        |      expr '+' expr
        |      expr '-' expr
        |      expr '*' expr
        |      expr '/' expr
        |      NAME
        ;
    
```

, poate fi folosita pentru a structura intrarea:

a = b = c * d - e - f * g , astfel: a = (b = ((c * d) - e) - (f * g))) .

Cind se foloseste acest mecanism, in general trebuie acordata o anumita precedenta si operatorilor unari, deoarece deseori un operator unar si unul binar pot avea aceeasi reprezentare simbolica, dar precedente diferite.

Un exemplu e '-'-ul unar si binar. Minus-ului unar i se poate acorda aceeasi precedenta ca si inmultirii, sau chiar mai mare, in timp ce minusul binar are o precedenta mai mica decit inmultirea. Cuvintul cheie %prec schimba nivelul de precedenta asociat cu o regula gramaticala particulara. %prec apare imediat dupa corpul regulii, inainte de actiune sau de punct si virgula de sfarsit al regulii. Cuvantul cheie %prec e urmat de un nume de token sau de un literal care va impune nivelul de precedenta. Precedenta regulii devine cea a numelui de token sau a literalului astfel specificat.

De exemplu, pentru ca minusul unar sa aiba aceeasi precedenta cu inmultirea, regulile pot sa arate astfel:

```

%left      '+' '-'
%left      '*' '/'
    
```

```

%%
expr :    expr '+' expr
      |    expr '-' expr
      |    expr '*' expr
      |    expr '/' expr
      |    '-' expr %prec '*'
      |    NAME
      ;

```

Un token declarat cu %left, %right sau %nonassoc nu mai trebuie declarat si cu %token. Precedentele si asociativitatile sint utilizate de YACC pentru a rezolva conflictele de analiza, si ele produc reguli de dezambiguizare. Formal, regulile lucreaza astfel:

1. Se inregistreaza precedentele si asociativitatile pentru acei token-i care le poseda.
2. Se asociaza o precedenta si o asociativitate cu fiecare regula gramaticala, si anume precedenta si asociativitatea ultimului token sau literal din corpul regulii. Daca este folosita constructia %prec, ea se suprapune peste aceasta situatie implicita. Unele reguli gramaticale pot sa nu aiba atasate precedenta si asociativitate.
3. Cand exista un conflict reducere-reducere sau deplasare-reducere, si fie simbolul de la intrare, fie regula gramaticala nu are precedenta si asociativitate, se folosesc cele doua reguli de dezambiguizare implicite si conflictele sunt raportate.
4. Daca exista un conflict deplasare-reducere si atit regula gramaticala cit si simbolul de la intrare au precedenta si asociativitate atasate, conflictul e rezolvat in favoarea actiunii ce este asociata cu o precedenta mai mare. Daca precedentele sint identice, atunci se foloseste asociativitatea. Asociativitatea stinga implica reducere, asociativitatea dreapta implica deplasare, iar non-asociativitatea implica eroare.

Conflictele rezolvate prin precedenta nu sint raportate. Aceasta inseamna ca erorile in specificarea precedentelor trebuie evitate. Fisierul y.output poate ilustra daca analizorul lucreaza corect.

2.8. Tratarea erorilor

Tratarea erorilor este un lucru dificil, multe din probleme fiind de natura semantica. Daca e intalnita o eroare, poate fi necesar, de exemplu, sa se memoreze arborele de derivare, sa se modifice sau sa se stearga intrari in tabela de simboluri si, in general, sa se seteze "switch"-uri pentru a se evita generarea de cod in continuare.

De obicei e util ca la intalnirea primei erori sa nu se abandoneze procesarea, ci sa se continue prelucrarea intrarii pentru a gasi eventual si alte erori de sintaxa. Aceasta conduce la problema de a face analizorul sa "restarteze" dupa o eroare.

O clasa generala de algoritmi pentru a face acest lucru implica ignorarea unui numar de token-i din intrare si incercarea de a ajusta analizorul astfel incit sa poata continua procesarea.

Pentru a permite utilizatorului controlul asupra acestui proces, YACC-ul considera numele de token "error" rezervat pentru tratarea erorilor. Acest token, folosit in reguli, marcheaza locurile unde pot sa apara erori si el poate realiza "refacere" sau "reluare".

Analizorul ia stari de pe stiva pina cind intra intr-o stare unde token-ul "error" e legal. Atunci se comporta ca si cand "error" ar fi token-ul "look-ahead" curent si realizeaza actiunea corespunzatoare. Token-ul "look-ahead" e resetat la token-ul ce a cauzat eroarea. Daca nu a fost specificata nici o regula speciala pentru eroare, procesarea se opreste la prima eroare detectata.

Pentru a preveni o cascada de mesaje de eroare, dupa detectarea unei erori, analizorul ramine in starea de eroare pina cind au fost cititi si deplasati cu succes trei token-i. Daca se detecteaza o noua eroare cand analizorul e deja in starea de eroare, nu se da nici un mesaj si token-ul din intrare e sters.

De exemplu, o regula de forma `stat : error` ar insemna de fapt, ca la o eroare de sintaxa, analizorul incearca sa sara peste instructiunea (`stat`) in care a fost vazuta eroarea. Mai exact, analizorul analizeaza inainte, cautand trei token-i ce ar putea urma legal unei instructiuni (`statement`) si incepe procesarea cu primul dintre acestia. Daca insa instructiunile nu sint suficient de distincte prin partea lor de inceput, analizorul poate lua un start fals, in mijlocul unei instructiuni, si sfarseste raportand o a doua eroare care de fapt nu exista.

Acestor reguli speciale li se pot asocia actiuni ce pot incerca sa reinitializeze tabele, sa reclame spatiu in tabela de simboluri, etc. Reguli de eroare ca cea prezentata anterior (`stat: error`) sunt foarte generale insa dificil de controlat.

Reguli mai simple pot fi cele de forma: `stat : error ';' .` In acest caz, daca exista o eroare, analizorul incearca sa sara peste instructiune (`stat`), citind pana la urmatorul `';`. Toti token-ii de dupa eroare si inainte de urmatorul `';` neputand fi deplasati, sunt ignorati.

Cand e vazut `';`, aceasta regula va fi redusa si se vor realiza actiunile de "curatire" asociate cu ea.

O alta forma a regulilor de eroare apare in aplicatiile interactive, unde poate fi de dorit ca dupa o eroare sa se permita reintroducerea liniei. O astfel de regula de eroare ar putea fi:

```
input : error '\n' { printf ("reenter last line:");} input
{ $$=$4;}
```

Exista insa o dificultate potentiala legata de aceasta solutie, si anume faptul ca analizorul trebuie sa proceseze corect trei token-i din intrare inainte de a admite ca s-a resincronizat corect. Daca linia nou introdusa contine o eroare in primii doi token-i, analizorul sterge token-ii respectivi fara sa dea nici un mesaj. Acest lucru e inacceptabil.

Din acest motiv, exista un mecanism ce poate fi folosit pentru a forta analizorul sa "creada" ca o eroare a fost complet acoperita. Este vorba de instructiunea **yyerrok**; , care, utilizata intr-o actiune, reseteaza analizorul la modul normal de lucru (il scoate fortat din starea de eroare). Astfel, ultimul exemplu poate fi scris in felul urmator:

```
input: error '\n'
{yyerrok;
printf ("Reenter last line: ");}
input
{ $$=$4;}
;
```

Dupa cum s-a mentionat, token-ul vazut imediat dupa simbolul "error" e token-ul de intrare la care a fost depistata eroarea. Uneori acest lucru poate fi nepotrivit. De exemplu, o actiune de acoperire a unei erori ar putea sa ia asupra sa responsabilitatea de a gasi locul corect pentru reluarea analizei. In acest caz, token-ul "look-ahead" precedent trebuie sters.

Instructiunea **yyclearin**; efectueaza acest lucru. Sa presupunem ca actiunea dupa eroare era de a apela o rutina de resincronizare sofisticata, furnizata de utilizator, care a incercat sa avanseze intrarea la inceputul urmatoarei instructiuni valide.

LAB-7-LFT- INSTRUMENTUL YACC

	Dupa ce aceasta rutina a fost apelata, urmatorul token returnat de <code>yylex()</code> este probabil primul token dintr-o instructiune legala. Vechiul token, ilegal, trebuie inlaturat si starea de eroare trebuie resetata. Acestea pot fi realizate astfel: <pre> stat : error { resync(); yyerrok; yyclearin; } ; </pre> Aceste mecanisme sint relativ dure, dar permit o refacere simpla a analizorului din multe erori. Mai mult, cind este necesar, utilizatorul poate obtine controlul pentru a se ocupa de actiunile de eroare cerute de alte portiuni din program.
2.9. Mediul YACC	
	Cind utilizatorul introduce o specificatie pentru YACC, iesirea este un fisier de programe C, numit in general ytab.c. Functia produsa de YACC se numeste yyparse() si returneaza un intreg. Ea utilizeaza repetat yylex(), analizorul lexical furnizat de catre utilizator, pentru a obtine token-i din intrare. Cind detecteaza o eroare ce nu poate fi acoperita, <code>yyparse()</code> returneaza valoarea 1. Pentru a obtine un program functional, utilizatorul trebuie sa-i furnizeze analizorului o anumita cantitate de "mediu". De exemplu, trebuie definit un program main, care eventual apeleaza <code>yyparse()</code>, si o rutina yyerror() care scrie un mesaj cind e detectata o eroare de sintaxa. Aceste doua rutine trebuie furnizate intr-o forma sau alta de catre utilizator. Pentru a usura efortul de utilizare a YACC-ului, acestuia i s-a atasat o biblioteca cu versiuni implicite de main si <code>yyerror()</code>. Aceste programe implicite sint foarte simple: <pre> main() { return(yyparse()); } #include <stdio.h> yyerror(char *s); { fprintf (stderr, "%s\n",s); } </pre> Argumentul lui <code>yyerror()</code> e un sir de caractere continind un mesaj de eroare (de obicei "syntax error"). In general, la detectarea unei erori de sintaxa programul afiseaza impreuna cu mesajul de eroare si numarul liniei din fisierul de intrare in care a aparut eroarea. Variabila intreaga external <code>yychar</code> contine numarul token-ului "look-ahead" din momentul detectarii erorii, si poate ajuta la punerea unor "diagnostice" mai bune. Variabila intreaga external yydebug , e setata in mod normal la valoarea zero. Daca ea are o alta valoare, analizorul produce si o descriere "verbose" a actiunilor sale, inclusiv informatii despre simbolul de intrare citit, si despre actiunile efectuate.
2.10. Pregatirea specificatiilor	
	Aceasta sectiune contine citeva sugestii pentru pregatirea unor specificatii clare si usor de modificat.
2.10.1. Stil pentru intrare	
	- Folositi litere mari pentru token-i si litere mici pentru non-terminale. - Puneti regulile gramaticale si actiunile pe linii separate pentru a permite modificarea lor separata. - Puneti toate regulile cu aceeasi parte stinga impreuna. Puneti partea stinga numai o data si scrieti toate regulile ce urmeaza incepand cu bara verticala. - Puneti punct si virgula ; numai dupa ultima regula cu o parte stanga data si pe o linie separata, permitand astfel adaugarea usoara de noi reguli.

e. Indentati corpul regulilor prin doua "tab"-uri.

2.10.2. Recursivitatea stanga

Analizorul folosit de YACC incurajeaza utilizarea in regulile gramaticale a recursivitatii stinga. Reguli de forma

```
list  :      item
      |      list ',' item
      ;
```

```
seq   :      item
      |      seq  item
      ;
```

, apar adesea cind se scriu specificatii de liste sau de secvente.

In aceste cazuri, prima regula va fi redusa numai pentru primul item, iar a doua regula va fi redusa pentru al doilea si toate cele ce ii urmeaza.

Cu reguli recursive dreapta, cum este de exemplu:

```
seq   :      item
      |      item seq
      ;
```

, rezulta un analizor mai mare si articolele sint vazute si reduse de la dreapta la stinga. Apare insa problema stivei interne a analizorului care e in pericol de a se "umple" cind din intrare e citita o secventa foarte lunga. De aceea, e recomandabil ca utilizatorul sa foloseasca recursivitatea stinga daca e posibil.

E bine sa se analizeze daca o secventa cu zero elemente are vreo semnificatie, si in cazul in care are, e indicat sa se includa in specificatia de secventa o regula vida, ca in urmatorul exemplu:

```
seq   :      /*empty*/
      |      seq  item
      ;
```

Prima regula va fi redusa intotdeauna o singura data, inainte ca primul item sa fie citit, iar a doua regula va fi redusa pentru fiecare item citit. Permitind secvente vide se creste generalitatea, dar pot sa apara conflicte daca YACC-ul trebuie sa decida ce secventa vida a "vazut", cind inca n-a citit destul ca sa stie acest lucru !

2.10.3. Probleme lexicale

Unele decizii lexicale depind de context. De exemplu, analizorul lexical ar putea dori sa stearga sau sa ignore spatiile, cu exceptia celor din interiorul sirurilor de caractere. Numele ce apar in declaratii, ar putea fi introduse intr-o tabela de simboluri, mai putin cele ce apar in expresii. O modalitate de a rezolva aceasta situatie este crearea unui "flag" global, examinat de analizorul lexical si setat de actiuni.

De exemplu, sa presupunem ca un program consta din zero sau mai multe declaratii, urmate de zero sau mai multe instructiuni:

```
%{
int    dflag;
}%
%%
prog   :      decls  stats
        ;
decls  :      /* empty */
        { dflag=1;}
        |      decls declaration
        ;
```

LAB-7-LFT- INSTRUMENTUL YACC

	<pre> stats : /* empty */ { dflag=0;} stats statement ; alte reguli Indicatorul dflag e zero cind se citesc instructiuni si 1 cind se citesc declaratii, exceptind primul token din prima instructiune. Pentru a putea spune ca sectiunea de declaratii s-a terminat, analizorul trebuie sa vada in intrare respectivul token. In multe cazuri, aceasta singura exceptie nu afecteaza analiza lexicala. </pre>
2.10.4. Cuvinte rezervate	
	Unele limbaje de programare permit utilizatorului sa foloseasca pentru etichete sau nume de variabile cuvinte (ca "if", "do", "else") care in mod normal sint rezervate, in ideea ca o asemenea utilizare nu intra in conflict cu folosirea legala a acestor nume in limbajul de programare. Acest lucru e greu de realizat in cadrul oferit de YACC. In general, e bine ca cuvintele cheie sa fie rezervate, deci sa nu se permita utilizarea lor ca nume de variabile.

3. Desfasurarea lucrarii

3.1. Se vor testa si analiza exemplele prezentate.

3.2. Se va studia modul de lucru al analizorului pentru exemplul dat la 2.5. si intrari incorecte ca DING DONG DONG, DING DONG, DING DONG DELL DELL, etc.

3.3. Se va analiza urmatorul exemplu, ce prezinta specificatia YACC corespunzatoare gramaticii discutate la lucrarea anterioara (varianta 1):

```

E -> T | E+T | E-T
T -> F | T*F | T/F
F -> n | (E) .

```

```

%term NUMAR
%left '+' '-'
%left '*' '/'
%%
lista :      /* vida */
        |      lista '\n'
        |      lista expr '\n'  { printf("\t%d\n", $2); }
        ;
expr :      NUMAR                { $$=$1; }
        |      expr '+' expr    { $$=$1+$3; }
        |      expr '-' expr    { $$=$1-$3; }
        |      expr '*' expr    { $$=$1*$3; }
        |      expr '/' expr    { $$=$1/$3; }
        |      '(' expr ')'      { $$=$2; }
        ;
%%

#include <stdio.h>
#include <ctype.h>
char *cmd;

```


LAB-7-LFT- INSTRUMENTUL YACC

```
main(argc, argv)
int argc;
char *argv[0];
{
    cmd=argv[0];
    yyparse();
}

yylex()
{
    int c;
    while(((c=getchar())!=' ') || (c=='\t')) ;
    if(c==EOF) return 0;
    if( isdigit(c) ) {
        ungetc(c,stdin);
        scanf("%d",&yylval);
        return NUMAR;
    }
    return c;
}

yyerror(s)    char* s;
{
    fprintf(stderr,"%s : %s",cmd,s);
    return;
}
```

3.4. Se va analiza urmatorul exemplu ce prezinta specificatia YACC pentru un mic calculator de birou, cu 26 de registri etichetati prin literele mici 'a' ... 'z', si care accepta expresii aritmetice alcatuite cu operatorii +, -, *, /, % (modulo), & (SI pe bit), | (SAU pe bit) si atribuirea. Daca o expresie are atribuire pe nivelul superior, valoarea nu este tiparita. In caz contrar, valoarea obtinuta e tiparita. Ca in C, daca un intreg incepe cu 0 e presupus a fi octal, altfel e presupus a fi zecimal. Exemplul arata cum sunt folosite precedentele si asociativitatile si prezinta o acoperire simpla a erorilor. Simplificarea majora e in faza de analiza lexicala, care e mult mai simpla decit in cazurile uzuale, iar iesirea e produsa imediat, linie cu linie. De notat modul in care intregii zecimali si octali sint cititi de catre regulile gramaticale, operatie care, probabil, ar putea fi mai bine facuta de catre analizorul lexical.

```
%{
#include <stdio.h>
#include <ctype.h>
int    regs[26];
int    baza;
%}

%start lista
%token    CIFRA        LITERA
%left    '|'
%left    '&'
%left    '+'    '-'
%left    '*'    '/'    '%'
%left    MINUSUNAR    /*furnizeaza precedenta pentru minus unar*/

%%    /*inceputul sectiunii pentru reguli*/
```

LAB-7-LFT- INSTRUMENTUL YACC

```
lista      :      /* vida */
            |      lista      instr      '\n'
            |      lista      error      '\n'          { yyerrok ; }
            ;
instr      :      expr          { printf("%d\n",$1); }
            |      LITERA '=' expr      { regs[$1]=$3; }
            ;
expr       :      '(' expr ')'      { $$=$2; }
            |      expr '+' expr      { $$=$1+$3; }
            |      expr '-' expr      { $$=$1-$3; }
            |      expr '*' expr      { $$=$1*$3; }
            |      expr '/' expr      { $$=$1/$3; }
            |      expr '%' expr      { $$=$1%$3; }
            |      expr '&' expr      { $$=$1&$3; }
            |      expr '|' expr      { $$=$1|$3; }
            |      '-' expr %prec MINUSUNAR { $$=-$2; }
            |      LITERA          { $$=regs[$1]; }
            |      numar
            ;
numar      :      CIFRA          { $$=$1;baza=($1==0) ? 8 : 10 ; }
            |      numar CIFRA      { $$=baza*$1+$2; }
            ;
```

```
%%      /* start programe */
```

```
yylex()
{      /*rutina analizor lexical          */
      /*returneaza LITERA pentru o litera mica          */
      /*yylval=0...25                                */
      /*returneaza CIFRA pentru o cifra , yylval=0...9*/
      /*toate celelalte caractere sint returnate imediat*/
      int      c;
      while( (c=getchar()) == ' ' ); /* ignora blancurile*/
      if(islower(c)) {      yynval=c-'a';      return(LITERA); }
      if(isdigit(c)) { yynval=c-'0';      return(CIFRA); }
      return(c);
}
```

4. Intrebari si dezvoltari

- 4.1. Sa se scrie specificatia YACC pentru o gramatica pentru expresii logice booleene.
- 4.2. Sa se propuna alte exemple si imbunatatiri la cele prezentate.